

In-Class Assignment: Set Up AWS DynamoDB and Connect Ticket Service

Project: AI-Powered Client Issue Triage Platform

Goal

In the previous assignment, we built a basic FastAPI Ticket API using an in-memory dictionary.

The current flow was:

```
Client
|
v
FastAPI Ticket API
|
v
In-memory dictionary
```

In this assignment, we will replace the in-memory repository with DynamoDB.

The new flow will be:

```
Client
|
v
FastAPI Ticket API
|
v
DynamoDB Tickets Table
```

By the end of this assignment, you should be able to:

1. Create or prepare an AWS account.
2. Create a DynamoDB table.
3. Configure AWS credentials locally.
4. Install Boto3.
5. Implement a DynamoDB repository for tickets.
6. Update the Ticket Service to use DynamoDB.
7. Test `POST /tickets` and `GET /tickets/{ticket_id}` with persistent storage.

Part 0: Starting Point

You should already have this project from the previous class:

```
ai-ticket-workflow-platform/  
  app/  
    __init__.py  
    main.py  
  
    models/  
      __init__.py  
      ticket_models.py  
  
    routers/  
      __init__.py  
      ticket_router.py  
  
    services/  
      __init__.py  
      ticket_service.py  
  
    repositories/  
      __init__.py  
      ticket_repository.py  
  
  requirements.txt  
  README.md
```

The previous repository used memory:

```
self.tickets = {}
```

That means:

```
If the server restarts, all tickets are lost.
```

Today we will store tickets in DynamoDB instead.

Part 1: AWS Account Setup

Step 1: Create an AWS account

Go to the AWS website and create an AWS account.

During account creation, you may need to provide:

- Email
- Password
- Account name
- Phone number
- Billing information
- Identity verification
- Support plan selection

For this project, the basic/free support plan is enough.

Step 2: Sign in to AWS Console

After the account is activated, sign in to the AWS Management Console.

Use the AWS Console to manage services such as:

- DynamoDB
- IAM
- SQS
- CloudWatch
- EC2

For today, we only need:

- DynamoDB
- IAM

Step 3: Set your AWS Region

At the top-right corner of the AWS Console, choose a region.

For this class, use:

```
us-west-2
```

This is the Oregon region.

Make sure you use the same region in both:

```
AWS Console  
Python code
```

In code, we will use:

```
region_name="us-west-2"
```

Part 2: Create IAM User for Local Development

Step 4: Why do we need IAM?

IAM stands for Identity and Access Management.

It controls:

```
Who can access AWS  
What services they can access  
What actions they can perform
```

Your Python code needs permission to talk to DynamoDB.

For local development, we will create an IAM user with programmatic access.

Step 5: Create an IAM user

In AWS Console:

```
Search for IAM  
Go to IAM
```

```
Click Users
Click Create user
```

User name:

```
ticket-platform-dev
```

For this class, this user will be used by your local Python application.

Step 6: Attach permissions

For a beginner class, you can attach this policy:

```
AmazonDynamoDBFullAccess
```

This gives the user permission to create, read, update, and delete DynamoDB resources.

In a real production system, we should use a more restrictive least-privilege policy.

Step 7: Create access key

After creating the user:

```
Open the user
Go to Security credentials
Create access key
Choose Local code
Create access key
```

Save:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
```

Do not commit these keys to GitHub.

Part 3: Configure AWS Credentials Locally

Step 8: Install AWS CLI

Check whether AWS CLI is installed:

```
aws --version
```

If it is not installed, install AWS CLI first.

Step 9: Configure AWS credentials

Run:

```
aws configure
```

Enter:

```
AWS Access Key ID: your_access_key
AWS Secret Access Key: your_secret_key
Default region name: us-west-2
Default output format: json
```

After this, your local machine can use AWS credentials when running Boto3 code.

Step 10: Verify AWS configuration

Run:

```
aws sts get-caller-identity
```

Expected output should include:

```
{
  "UserId": "...",
  "Account": "...",
```

```
"Arn": "..."  
}
```

If this command works, your credentials are configured correctly.

Part 4: Create DynamoDB Table

Step 11: Open DynamoDB Console

In AWS Console:

```
Search for DynamoDB  
Open DynamoDB  
Click Tables  
Click Create table
```

Step 12: Create Tickets table

Use the following table configuration:

```
Table name: Tickets  
Partition key: ticket_id  
Partition key type: String  
Sort key: None  
Table settings: Default settings  
Capacity mode: On-demand
```

Click:

```
Create table
```

Wait until table status becomes:

```
Active
```

Step 13: Understand the table design

Our main access pattern is:

```
Get one ticket by ticket_id
```

That is why we use:

```
ticket_id
```

as the partition key.

Example item:

```
{
  "ticket_id": "ticket_ab12cd34",
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app",
  "status": "CREATED",
  "created_at": "2026-07-29T10:00:00Z",
  "updated_at": "2026-07-29T10:00:00Z"
}
```

Part 5: Update Python Dependencies

Step 14: Add Boto3

Open `requirements.txt`.

Add:

```
boto3
```

The file should now look like:

```
fastapi
uvicorn
```

```
pydantic
boto3
```

Install dependencies:

```
pip install -r requirements.txt
```

Part 6: Update Ticket Models

Step 15: Update app/models/ticket_models.py

Replace the file with:

```
from enum import Enum
from typing import Optional
from pydantic import BaseModel, Field

class TicketStatus(str, Enum):
    CREATED = "CREATED"
    WORKFLOW_SELECTED = "WORKFLOW_SELECTED"
    WORKFLOW_CREATED = "WORKFLOW_CREATED"
    WORKFLOW_QUEUED = "WORKFLOW_QUEUED"
    WORKFLOW_RUNNING = "WORKFLOW_RUNNING"
    RESOLVED = "RESOLVED"
    FAILED = "FAILED"

class CreateTicketRequest(BaseModel):
    user_id: str = Field(..., min_length=1)
    message: str = Field(..., min_length=5)
    source: Optional[str] = "unknown"

class UpdateTicketStatusRequest(BaseModel):
    status: TicketStatus

class TicketResponse(BaseModel):
    ticket_id: str
    user_id: str
    message: str
```

```
source: str
status: TicketStatus
created_at: str
updated_at: str
```

Part 7: Create DynamoDB Ticket Repository

Step 16: Rename old repository

The previous repository was in:

```
app/repositories/ticket_repository.py
```

You can replace it directly with the DynamoDB version.

Step 17: Implement DynamoDB repository

Edit:

```
app/repositories/ticket_repository.py
```

Add:

```
from typing import List, Optional

import boto3
from botocore.exceptions import ClientError

from app.models.ticket_models import TicketResponse, TicketStatus

class TicketRepository:
    def __init__(self):
        dynamodb = boto3.resource("dynamodb", region_name="us-west-2")
        self.table = dynamodb.Table("Tickets")

    def save(self, ticket: TicketResponse) -> TicketResponse:
        self.table.put_item(
            Item={
```

```

        "ticket_id": ticket.ticket_id,
        "user_id": ticket.user_id,
        "message": ticket.message,
        "source": ticket.source,
        "status": ticket.status.value,
        "created_at": ticket.created_at,
        "updated_at": ticket.updated_at,
    }
)

return ticket

def get_by_id(self, ticket_id: str) -> Optional[TicketResponse]:
    response = self.table.get_item(
        Key={
            "ticket_id": ticket_id
        }
    )

    item = response.get("Item")

    if item is None:
        return None

    return self._item_to_ticket(item)

def list_all(self) -> List[TicketResponse]:
    response = self.table.scan()
    items = response.get("Items", [])

    return [
        self._item_to_ticket(item)
        for item in items
    ]

def update_status(
    self,
    ticket_id: str,
    status: TicketStatus,
    updated_at: str,
) -> Optional[TicketResponse]:
    try:
        response = self.table.update_item(
            Key={
                "ticket_id": ticket_id
            },
            UpdateExpression="SET #status = :status, updated_at
= :updated_at",

```

```

        ExpressionAttributeNames={
            "#status": "status"
        },
        ExpressionAttributeValues={
            ":status": status.value,
            ":updated_at": updated_at,
        },
        ReturnValues="ALL_NEW",
    )

    updated_item = response.get("Attributes")

    if updated_item is None:
        return None

    return self._item_to_ticket(updated_item)

except ClientError:
    return None

def _item_to_ticket(self, item: dict) -> TicketResponse:
    return TicketResponse(
        ticket_id=item["ticket_id"],
        user_id=item["user_id"],
        message=item["message"],
        source=item.get("source", "unknown"),
        status=TicketStatus(item["status"]),
        created_at=item["created_at"],
        updated_at=item["updated_at"],
    )

```

Part 8: Update Ticket Service

Step 18: Update app/services/ticket_service.py

Replace the file with:

```

from datetime import datetime, timezone
from typing import List, Optional
from uuid import uuid4

from app.models.ticket_models import (
    CreateTicketRequest,

```

```

        TicketResponse,
        TicketStatus,
    )
    from app.repositories.ticket_repository import TicketRepository

class TicketService:
    def __init__(self):
        self.ticket_repository = TicketRepository()

    def create_ticket(self, request: CreateTicketRequest) -> TicketResponse:
        now = self._current_time_iso()
        ticket_id = f"ticket_{uuid4().hex[:8]}"

        ticket = TicketResponse(
            ticket_id=ticket_id,
            user_id=request.user_id,
            message=request.message,
            source=request.source or "unknown",
            status=TicketStatus.CREATED,
            created_at=now,
            updated_at=now,
        )

        return self.ticket_repository.save(ticket)

    def get_ticket(self, ticket_id: str) -> Optional[TicketResponse]:
        return self.ticket_repository.get_by_id(ticket_id)

    def list_tickets(self) -> List[TicketResponse]:
        return self.ticket_repository.list_all()

    def update_ticket_status(
        self,
        ticket_id: str,
        status: TicketStatus,
    ) -> Optional[TicketResponse]:
        now = self._current_time_iso()

        return self.ticket_repository.update_status(
            ticket_id=ticket_id,
            status=status,
            updated_at=now,
        )

    def _current_time_iso(self) -> str:
        return datetime.now(timezone.utc).isoformat()

```

```
ticket_service = TicketService()
```

Part 9: Update Ticket Router

Step 19: Update app/routers/ticket_router.py

Replace the file with:

```
from typing import List

from fastapi import APIRouter, HTTPException, status

from app.models.ticket_models import (
    CreateTicketRequest,
    TicketResponse,
    UpdateTicketStatusRequest,
)
from app.services.ticket_service import ticket_service

router = APIRouter(
    prefix="/tickets",
    tags=["Tickets"],
)

@router.post(
    "",
    response_model=TicketResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_ticket(request: CreateTicketRequest):
    return ticket_service.create_ticket(request)

@router.get(
    "",
    response_model=List[TicketResponse],
)
def list_tickets():
    return ticket_service.list_tickets()
```

```

@router.get(
   ("/{ticket_id}",
    response_model=TicketResponse,
)
def get_ticket(ticket_id: str):
    ticket = ticket_service.get_ticket(ticket_id)

    if ticket is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Ticket not found",
        )

    return ticket

@router.patch(
   ("/{ticket_id}/status",
    response_model=TicketResponse,
)
def update_ticket_status(
    ticket_id: str,
    request: UpdateTicketStatusRequest,
):
    ticket = ticket_service.update_ticket_status(
        ticket_id=ticket_id,
        status=request.status,
    )

    if ticket is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Ticket not found",
        )

    return ticket

```

Part 10: Verify app/main.py

Step 20: Check app/main.py

Make sure it looks like:

```
from fastapi import FastAPI

from app.routers.ticket_router import router as ticket_router

app = FastAPI(
    title="AI Ticket Workflow Platform",
    description="A backend platform for client issue triage and workflow automation.",
    version="0.1.0",
)

app.include_router(ticket_router)

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }
```

Part 11: Run the Server

Step 21: Start FastAPI server

Run:

```
uvicorn app.main:app --reload
```

Open Swagger UI:

```
http://127.0.0.1:8000/docs
```

Part 12: Test the APIs

Test 1: Health Check

Call:

```
GET /health
```

Expected response:

```
{
  "status": "ok"
}
```

Test 2: Create Ticket

Call:

```
POST /tickets
```

Request body:

```
{
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "ticket_id": "ticket_ab12cd34",
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app",
  "status": "CREATED",
  "created_at": "2026-07-29T...",
}
```

```
"updated_at": "2026-07-29T..."
}
```

The actual `ticket_id` and timestamps will be different.

Test 3: Check DynamoDB Console

Go to:

```
AWS Console
DynamoDB
Tables
Tickets
Explore table items
```

You should see the ticket item you just created.

Test 4: Get Ticket By ID

Use the ticket ID from Test 2.

Call:

```
GET /tickets/{ticket_id}
```

Example:

```
GET /tickets/ticket_ab12cd34
```

Expected response:

```
{
  "ticket_id": "ticket_ab12cd34",
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app",
  "status": "CREATED",
  "created_at": "2026-07-29T..."
}
```

```
"updated_at": "2026-07-29T..."
}
```

Test 5: Restart Server and Get Ticket Again

Stop the server.

Then restart it:

```
uvicorn app.main:app --reload
```

Call:

```
GET /tickets/{ticket_id}
```

The ticket should still exist.

This proves that we are no longer using in-memory storage.

Test 6: List Tickets

Call:

```
GET /tickets
```

Expected response:

```
[
  {
    "ticket_id": "ticket_ab12cd34",
    "user_id": "user_123",
    "message": "TSLA price has not updated for 30 minutes",
    "source": "mobile_app",
    "status": "CREATED",
    "created_at": "2026-07-29T...",
    "updated_at": "2026-07-29T..."
  }
]
```

Note:

This API uses scan.
Scan is okay for this class project, but it is not ideal for large production tables.

Test 7: Update Ticket Status

Call:

```
PATCH /tickets/{ticket_id}/status
```

Request body:

```
{  
  "status": "RESOLVED"  
}
```

Expected response:

```
{  
  "ticket_id": "ticket_ab12cd34",  
  "user_id": "user_123",  
  "message": "TSLA price has not updated for 30 minutes",  
  "source": "mobile_app",  
  "status": "RESOLVED",  
  "created_at": "2026-07-29T...",  
  "updated_at": "2026-07-29T..."  
}
```

Part 13: Common Issues

Issue 1: Unable to locate credentials

Error message may look like:

```
Unable to locate credentials
```

Fix:

```
aws configure
```

Then enter your access key, secret key, region, and output format.

Issue 2: ResourceNotFoundException

This usually means the table name or region is wrong.

Check:

```
Table name should be Tickets  
Region should be us-west-2
```

Your code should use:

```
region_name="us-west-2"
```

Issue 3: AccessDeniedException

This usually means your IAM user does not have DynamoDB permission.

Check that the IAM user has:

```
AmazonDynamoDBFullAccess
```

Issue 4: Validation error from FastAPI

If the request body is missing required fields, FastAPI will return a validation error.

For example, this is invalid:

```
{  
  "user_id": "user_123"  
}
```

because `message` is required.

Part 14: Git Commit and Push

Step 22: Check git status

```
git status
```

Step 23: Add files

```
git add .
```

Step 24: Commit

```
git commit -m "Connect ticket service to DynamoDB"
```

Step 25: Push

```
git push origin main
```

If your branch is named `master`, run:

```
git push origin master
```

Part 15: Update README

Add this section to `README.md`:

DynamoDB Integration

The Ticket API now stores tickets in AWS DynamoDB instead of in-memory storage.

DynamoDB Table

Table name:

``text

Tickets

Partition key:

ticket_id

Supported APIs

POST /tickets

GET /tickets

GET /tickets/{ticket_id}

PATCH /tickets/{ticket_id}/status

How to Run

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

AWS Setup

Before running the app, configure AWS credentials:

aws configure

Use region:

us-west-2

Acceptance Criteria

Your assignment is complete if:

1. You created or prepared an AWS account.
2. You created an IAM user for local development.
3. You configured AWS credentials locally.
4. You created a DynamoDB table named `Tickets`.
5. The `Tickets` table uses `ticket\_id` as the partition key.
6. `POST /tickets` writes a ticket to DynamoDB.
7. `GET /tickets/{ticket\_id}` reads a ticket from DynamoDB.
8. `GET /tickets` lists tickets from DynamoDB.
9. `PATCH /tickets/{ticket\_id}/status` updates ticket status in DynamoDB.
10. Tickets still exist after restarting the FastAPI server.
11. The code is pushed to GitHub.

Questions to Answer

Please answer these questions after finishing the assignment:

1. What problem did DynamoDB solve compared to in-memory storage?
2. What is the partition key of the `Tickets` table?
3. Why do we use `ticket\_id` as the partition key?
4. What does `put\_item` do?
5. What does `get\_item` do?
6. What does `update\_item` do?
7. Why is `scan` not ideal for large production tables?
8. What is IAM used for?
9. Why should we not commit AWS credentials to GitHub?
10. What will we add next after DynamoDB?

Next Class Preview

In the next class, we will start connecting the ticket system to workflow processing.

The next flow will be:

```
```text
POST /tickets
|
v
Create ticket in DynamoDB
|
```

```
 v
Select workflow
|
 v
Create workflow run
|
 v
Send workflow_run_id to SQS
|
 v
Workflow worker consumes message
```

We will introduce:

```
WorkflowDefinition
WorkflowRun
SQS
Worker
```